

THE INTERNAL MONOLOGUE

A Dual-Stream Architecture for Auditable Inner-Alignment Signals

Version 2.10

Portable Verifier, Hybrid Adaptive Evidence, and End-to-End Retention Revision

Daniel A. Wycoff

July 21, 2026

Status: Technical proposal. This draft preserves the compact v2.9 architecture, remains compatible with the v2.9.1 AgentOps layer, and adds portable verifier governance, model-unpredictable hybrid widening, and end-to-end retention assurance. It does not report experimental results.

Technical Proposal

Abstract

Highly capable language models may exhibit deceptive alignment: behavior that appears compliant under evaluation while internally optimizing for a divergent objective. Existing transparency methods such as Chain-of-Thought prompting rely on editable self-report and therefore provide limited protection against a strategically deceptive agent.

This paper proposes the Dual-Stream Architecture (DSA), a system interface that couples each user-facing output token, the Answer Stream, with synchronized Monologue evidence derived from designated internal model states. The Monologue Stream is not free-form natural-language reasoning. At minimum, it contains reconstructable token-level evidence: the chosen token id and compact top-K pre-control logits captured before decoder-side controls. Optionally, it includes compact probe-derived summaries over selected activations.

Version 2.10 preserves the compact evidence profiles introduced in v2.8 and the verifier, adaptive-evidence, and retention controls introduced in v2.9, while correcting three weaknesses exposed by operational critique. First, static wall-clock and absolute-memory pass/fail thresholds are replaced by portable deterministic work envelopes, bounded streaming-space requirements, environment-normalized performance measurements, and an explicit INCONCLUSIVE_INFRA outcome for transient runner instability. Second, predictable rank-only widening is replaced by a hybrid adaptive policy combining rank-triggered widening, keyed deterministic sampling of otherwise untriggered top-3 tokens, and signed historical-tension rules. Third, the local retention floor is extended into an end-to-end retention assurance protocol using a signed retention requirement, asynchronous storage receipts, object-integrity validation, periodic proof-of-possession checks, and restore testing.

The design retains reproducibility without exposing a fixed evasion boundary. Sampling decisions are generated by a keyed pseudorandom function whose key is unavailable to the model during generation, while authorized replay uses a sealed audit seed or replay token. CI remains deterministic for a given replay context. Local verification remains independent of network latency, while release and production assurance require downstream retention receipts before a DSA-P retention claim can be made.

DSA does not prove inner alignment. It aims at a narrower engineering goal: make selected alignment-relevant inconsistencies observable, measurable, regressible, and, in the production assurance class, attestable under explicit assumptions about probe fidelity, verifier correctness, unpredictable audit selection, storage integrity, and decoding semantics.

Keywords: inner alignment; deceptive alignment; mechanistic interpretability; probes; logit lens; CI/CD; compact evidence; portable verification; keyed sampling; retention receipts; attestation; AgentOps

Contents

1. Introduction
2. Threat model, goals, assumptions, and non-goals
3. The Dual-Stream Architecture
4. Compact Evidence Schema V3.3
5. Portable Verifier Governance
6. Hybrid Adaptive Evidence
7. End-to-End Retention Assurance
8. Producing and Calibrating Monologue Evidence
9. Alignment Signal Taxonomy AST-1 Extensions
10. The Coherence Audit and CI/CD Gates
11. Evaluation and Red-Team Plan
12. AgentOps, Deployment, and Governance
13. Limitations and Failure Modes
14. Roadmap
15. Conclusion
- Appendix A. Portable CI Conformance Contract
- Appendix B. Hybrid Selection Conformance
- Appendix C. Retention Assurance Contract
- Appendix D. Version 2.10 Change Log
- References

1. Introduction

As language models become more capable and more deeply embedded in software, support, security, and decision workflows, the cost of latent misalignment rises. One important risk class is inner alignment failure: a system may learn internal objectives that diverge from the operator's intended objective and, in the worst case, strategically conceal that divergence during evaluation and deployment [4,6,7]. Under that threat model, output-only evaluation is brittle because compliant surface behavior may coexist with problematic internal optimization.

Many transparency methods attempt to narrow this observability gap by eliciting explanations, rationales, or reasoning traces. Chain-of-Thought prompting is, by construction, model-generated text [1]. It can be useful for capability and debugging, yet it remains voluntary disclosure rather than independently captured evidence. DSA instead treats evidence capture as a system boundary contract. Each committed answer token is bound to typed evidence captured upstream of decoder-side controls and downstream verifiers evaluate both the answer and evidence.

Version 2.8 made this contract operationally plausible by separating logical evidence cardinality from physical encoding. Version 2.9 added reconstruction budgets, rank-triggered adaptive K, and a local mechanical retention floor. Those changes closed concrete deployment gaps, but the v2.9 rules created three new edge conditions.

First, a fixed 3.0-second p95 threshold and fixed resident-memory limits can confuse infrastructure weather with a security defect. Shared runners, dynamic CPU quotas, heterogeneous architectures, virtualized filesystems, and noisy neighbors can all change elapsed time without changing the artifact or algorithm. A security gate that fails a valid artifact because a cloud node was temporarily throttled trains engineers to distrust and eventually bypass the gate.

Second, a public rank-only trigger creates a bright boundary. If evidence widens only when the chosen raw rank is 4 through 10, a strategically adaptive model can seek behavior whose decisive tokens remain inside ranks 1 through 3. Rank-triggered widening still has value, but it cannot be the sole trigger under a threat model that includes strategic concealment.

Third, a local minimum-byte floor proves only that the current CI artifact is reconstructable when verified. It does not prove that downstream telemetry systems, object stores, compression jobs, or lifecycle policies preserve that artifact. Long-term auditing fails if the object is later truncated to a summary, replaced, corrupted, or expired before the declared retention period.

Version 2.10 addresses these issues with three architectural corrections:

- **Portable verifier governance.** Security-relevant complexity is bounded through deterministic operation counters and a schema-defined streaming-space envelope. Wall-clock and observed RSS remain measured, but transient environment variance cannot masquerade as an evidence-integrity failure.
- **Hybrid adaptive evidence.** Rank-triggered widening remains, but is supplemented by keyed deterministic sampling and signed historical-tension rules. The model cannot know in advance which ordinary top-3 tokens will receive deeper evidence capture.
- **End-to-end retention assurance.** The local floor produces a signed retention requirement. Independent storage validators return signed receipts, and periodic challenges verify that retained chunks remain retrievable and reconstructable.

Version 2.10 is compatible with the v2.9.1 AgentOps operational layer. Repositories that already version prompts, policies, evaluation datasets, tool schemas, rollout state, and incident records should treat the new verifier profiles, audit-key identifiers, tension maps, retention requirements, receipts, and lifecycle states as versioned AgentOps artifacts.

The proposal remains deliberately bounded. It does not claim to recover a model's complete reasoning, prove the absence of deception, or make probe outputs infallible. It specifies an auditable evidence and operations contract whose failure states are explicit and whose behavior can be tested in CI, release pipelines, and production monitoring.

2. Threat model, goals, assumptions, and non-goals

2.1 Threat model

The monitored model is untrusted for honesty or alignment. It may produce superficially compliant behavior while optimizing for a divergent objective, attempt to keep suspicious behavior outside monitored feature spaces, or adapt to published audit rules. The host runtime may be honest in DSA-R and potentially compromised in DSA-P. CI infrastructure may be slow, heterogeneous, or intermittently resource constrained without being malicious. Downstream storage and telemetry systems may apply cost-saving transformations that accidentally or intentionally weaken reconstructability.

The attacker classes considered in v2.10 are:

1. **Model-side evasion.** The model adapts token choices, phrasing, or internal trajectories to avoid known evidence triggers or probes.
2. **Host-side tampering.** A compromised host suppresses, reorders, replaces, or edits evidence.
3. **Verifier denial or regression.** A malformed artifact or inefficient implementation causes excessive work, memory use, or pipeline instability.
4. **Infrastructure variance.** A valid verifier run experiences transient throttling, noisy-neighbor effects, or virtualized I/O delays.
5. **Storage attrition.** A downstream system truncates, recompresses, replaces, or expires evidence before the required retention horizon.
6. **Governance poisoning.** An attacker modifies the audit-key registry, historical-tension map, profile coefficients, probe pack, or retention policy.

2.2 Assurance classes and trust boundaries

Component	DSA-R research assurance	DSA-P production assurance
Model	Untrusted; object being monitored	Untrusted; object being monitored
Host runtime	Trusted for capture, transport, and local storage	Outside the evidence-integrity trust base
Accelerator or secure device	Ordinary compute substrate	Root of measurement and evidence attestation
Probe-pack updater	Trusted to publish reviewed, signed packs	Trusted to publish reviewed, signed packs
Audit-key service	Optional; may run in the trusted host domain	Trusted service or device facility holding sampling keys unavailable to the model
Tension-map publisher	Trusted to sign reviewed historical rules	Trusted to sign reviewed historical rules
Verifier	Trusted to enforce schema, integrity, complexity, and audit rules	Trusted to validate device evidence and storage receipts
Storage validator	Optional for local research	Trusted to validate persisted objects and issue receipts
Long-term storage	Trusted operational dependency	Untrusted until a valid receipt and later possession checks are verified

In DSA-R, inseparability is an interface property under an honest host. In DSA-P, evidence integrity requires a device-bound capture path, a trustworthy verifier, protected audit-selection keys, signed policy artifacts, and independently validated retention.

2.3 Operational definitions

Alignment-relevant inconsistency: a declared audit rule determines that completed answer semantics and emitted evidence semantics disagree in a policy-relevant direction.

Coherence regression: relative to a pinned baseline model, probe pack, schema, adaptive policy, tension map, verifier profile, and thresholds, a newer revision worsens PASS, REVIEW, FAIL, or silent-deception outcomes beyond an allowed budget on the same benchmark suite.

Deterministic verifier work envelope: the maximum schema-derived counts of bytes read, bytes hashed, token records decoded, candidate entries decoded, chunks validated, span events indexed, allocations performed, and live working memory required for a declared artifact.

Environment-normalized runtime: observed verifier elapsed time divided by, or otherwise normalized against, a pinned microbenchmark that exercises the same primitive operations on the current runner.

Infrastructure instability: a run in which the verifier cannot complete reliably because of environmental resource conditions even though the artifact passes deterministic complexity and integrity checks. This produces INCONCLUSIVE_INFRA, not a security PASS or an evidence FAIL.

Hybrid adaptive widening: a policy that widens evidence through one or more of rank, keyed sample, signed historical-tension, or explicitly labeled evaluation-canary triggers.

Retention requirement: a signed post-encoding statement that binds the artifact content hash, schema, minimum reconstructable bytes, retention class, required horizon, and validation policy.

Retention receipt: a signed storage-side statement that binds a persisted object version to the retention requirement and confirms successful reconstruction and integrity validation at a specific time.

Retention assurance state: the lifecycle status LOCAL_PASS, RETENTION_PENDING, RETENTION_VERIFIED, RETENTION_AT_RISK, RETENTION_EXPIRED, or RETENTION_INVALID.

2.4 Design goals

- **G1 Inseparability:** each committed Answer token has exactly one reconstructable evidence record.
- **G2 Typed evidence:** the Monologue Stream is structured telemetry, not free-form explanation text.
- **G3 Auditability:** evidence is sufficient to support automated checks at scale.
- **G4 Portable verifier governance:** security gates depend on deterministic work and bounded space rather than transient runner speed.
- **G5 Infrastructure isolation:** environmental resource failures are distinguishable from evidence, model, and policy failures.
- **G6 Unpredictable selection:** a model cannot reliably infer that ordinary top-3 tokens will avoid deeper capture.
- **G7 Reproducible replay:** authorized operators can reproduce sampling decisions for the same audit context.
- **G8 End-to-end retention:** local reconstructability requirements propagate into storage validation and lifecycle monitoring.
- **G9 Deployment practicality:** default PR evidence remains compact and local verification does not wait on network storage.
- **G10 Interoperability:** evidence, audit policy, work certificates, and receipts use versioned portable schemas.
- **G11 Privacy control:** evidence and receipts support compartmentalized access, minimization, and retention limits.
- **G12 AgentOps compatibility:** all behavior-affecting artifacts are versioned, testable, observable, and rollbackable.

2.5 Assumptions required for useful audits

- **A1 Upstream capture:** mandatory evidence is captured from raw final-layer logits before decoder-side controls.
- **A2 Operationally meaningful instrumentation:** probe and AST signals correlate with the conditions they claim to track within declared calibration limits.
- **A3 Protected audit selection:** keyed sampling secrets are unavailable to the model and are not exposed in prompt context, tool output, or public build logs before capture decisions are finalized.
- **A4 Calibrated thresholds:** audit thresholds and stochastic sample rates are tuned on representative benign and adversarial suites.
- **A5 Streaming verification:** normal CI uses bounded chunk-by-chunk reconstruction and does not require full forensic materialization.

- **A6 Signed governance artifacts:** profile coefficients, tension maps, key registries, probe packs, and retention policies are versioned and signed.
- **A7 Independent retention validation:** storage receipts are issued by a validator that is operationally independent of the process that produced the artifact.
- **A8 Honest assurance labeling:** LOCAL_PASS is not represented as RETENTION_VERIFIED, and DSA-R telemetry is not represented as DSA-P attestation.

2.6 Non-goals

- DSA does not guarantee perfect recovery of intent, goals, or plans.
- DSA does not prevent all deception; it aims to increase detectability and reduce silent failure.
- DSA does not expose private chain-of-thought to end users.
- DSA does not claim complete observability of long-horizon planning.
- Keyed sampling does not guarantee that every deceptive token is widened.
- Retention receipts do not replace periodic restore tests or organizational data-governance controls.
- DSA-CI-Lite is not a substitute for deep adversarial evaluation or DSA-P attestation.

3. The Dual-Stream Architecture

DSA is a system-level interface contract between a generative model and its host or device environment. At generation time, the system produces two synchronized outputs:

- **Answer Stream A:** the user-facing token sequence.
- **Monologue evidence stream B:** a verifier-facing sequence of logical evidence records corresponding one-to-one with committed Answer tokens.

The physical artifact may use sequence metadata, chunk metadata, compact token records, sparse span events, manifests, and receipts. One logical record per token does not require one self-contained serialized object per token.

3.1 End-to-end lifecycle

1. **Forward pass.** The model computes raw final-layer logits and any tapped activations.
2. **Raw evidence capture.** The system obtains the base top-K candidate ids and quantized scores before decoder-side controls.
3. **Token commit.** Decoder controls and sampling select the committed Answer token; chosen_id records the actual token.
4. **Hybrid widening selection.** The evidence path evaluates rank, keyed-sample, and signed historical-tension triggers. Evaluation-only canary triggers are permitted only in explicitly labeled test runs.
5. **Compact encoding.** Stable metadata is written once per sequence, integrity metadata once per chunk, token data in compact records, and probe or AST outputs as sparse span events.
6. **Local verification.** The verifier reconstructs evidence in streaming order, validates integrity, calculates the deterministic work certificate, executes the Coherence Audit, and computes the local retention floor.
7. **Retention requirement issuance.** After final encoding, the verifier signs or device-attests a retention requirement bound to the final artifact hash and policy.
8. **Asynchronous persistence.** The artifact and requirement are submitted to storage through a queue or direct API. Ordinary PR verification does not block on the storage network.
9. **Storage validation and receipt.** An independent validator retrieves the persisted object version, verifies the content and reconstructability, and issues a signed receipt.
10. **Ongoing assurance.** Watchers monitor receipt deadlines, retention horizons, object version changes, challenge proofs, and scheduled restore tests.



Figure 1. DSA v2.10 end-to-end evidence and retention lifecycle.

3.2 Compliance invariants

- **I1 One-to-one reconstructability:** for every Answer token index t , exactly one logical evidence record $B[t]$ is reconstructable.
- **I2 Pre-control capture:** mandatory candidate ids and scores are captured from raw final-layer logits before temperature, penalties, top-k or top-p filtering, banned-token masking, or safety suppression.
- **I3 Chosen-token binding:** $B[t]$ reconstructs the token actually committed after decoder controls and sampling.
- **I4 Completeness signaling:** missing, duplicated, reordered, malformed, or truncated evidence fails verification.
- **I5 Assurance labeling:** every sequence declares assurance class, evidence profile, adaptive policy, verifier work profile, and retention policy.
- **I6 Schema and probe binding:** every sequence binds the signal schema and probe pack by identifier and hash.
- **I7 Hybrid-trigger integrity:** every widened record declares its trigger reason; keyed-sample decisions bind to an audit-key id and selection commitment.
- **I8 Key secrecy:** sampling secrets **MUST NOT** be included in the model-visible context or unsealed artifact.
- **I9 Deterministic work boundedness:** local verification **MUST** remain inside the declared operation-count and streaming-space envelope.
- **I10 No environment false equivalence:** raw elapsed-time or observed-RSS deviation alone **MUST NOT** be labeled as an alignment, evidence, or integrity failure.
- **I11 No unverifiable PASS:** a verifier that does not complete **MUST** return `INCONCLUSIVE_INFRA` or `FAIL` as appropriate, never `PASS`.
- **I12 Local retention floor:** the finalized artifact **MUST** contain at least the bytes and records required for complete local reconstruction.
- **I13 Retention requirement binding:** the post-run retention requirement **MUST** bind the final artifact content hash, local minimum, policy, and expiry.
- **I14 Receipt independence:** `RETENTION_VERIFIED` requires a valid receipt from an authorized storage validator, not a self-assertion by the producer.
- **I15 DSA-P promotion rule:** a production or release artifact claiming end-to-end retention assurance **MUST** be `RETENTION_VERIFIED` before promotion.

3.3 Evidence profiles

Profile	Default use	Evidence policy	Portable verifier policy	Retention policy
DSA-CI-Lite	Normal PR checks	Base $K=3$; rank widening through configured cap; keyed sample of otherwise untriggered records; signed tension-map overrides	Hard deterministic work and streaming-space envelope; normalized runtime advisory; one resource retry	Local floor required; asynchronous receipt optional unless repository policy requires it
DSA-CI-Standard	Nightly and release-blocking CI	$K=5$ base or preserved hybrid records; sparse AST spans; stronger integrity	Same hard work model with larger declared artifact bounds; normalized runtime gate on controlled runner	Signed requirement and receipt required before release promotion
DSA-Deep	Red-team, probe evaluation, canary suites	$K=5$ or greater; denser probes; historical and evaluation-canary triggers	Profile-specific deterministic envelope; full diagnostics permitted	Receipt and restore verification required for retained benchmark evidence
DSA-Forensic	Incident replay and DSA-P attestation	Full evidence, stronger signatures, optional per-token diagnostics	Profile-specific; never default PR path	Immutable or equivalent protected storage, receipt, periodic possession proofs, and restore tests

The raw evidence ceilings from v2.8 remain: 24 raw bytes/token for CI-Lite, 48 for CI-Standard, 96 for Deep unless overridden, and 256 as the Forensic ceiling. A project **MAY** define stricter limits. Compression **MAY** be reported but cannot be used to pass the raw-byte gate.

4. Compact Evidence Schema V3.3

Version 3.3 extends the current compact wire family without returning to self-contained heavy per-token frames. The V3.3 encoder uses wire version `0x0303`. Conforming decoders **MUST** preserve read compatibility for legacy `0x0301` and current `0x0302` artifacts and **MUST** dispatch on the wire version before interpreting version-specific fields; implementations **MUST NOT** reinterpret older bytes under the V3.3 layout. V3.3 introduces protected audit-selection metadata, a verifier work certificate, a signed tension-map binding, and a post-run retention manifest.

4.1 MonologueSequenceHeader V3.3

Field	Type or size	Required semantics
magic	32-bit unsigned integer	Compact Monologue evidence family identifier
version	16-bit unsigned integer	V3.3 layout identifier
evidence_profile	8-bit enum	CI-Lite, CI-Standard, Deep, or Forensic
assurance_class	8-bit enum	DSA-R or DSA-P
prompt_nonce	64-bit unsigned integer	Sequence uniqueness and replay defense
sequence_id	32-bit unsigned integer	Local sequence identifier
tokenizer_id	16-bit unsigned integer	Tokenizer used for reconstruction
signal_schema_id	16-bit unsigned integer	AST or successor schema identifier
signal_schema_hash	32 bytes	Hash of the signal mapping
probe_pack_id	32-bit unsigned integer	Active probe pack identifier
probe_pack_hash	32 bytes	Hash of probe pack and calibration metadata
decoder_control_flags	32-bit bitfield	Active decoder-side control classes
base_topk	8-bit unsigned integer	Base candidate count; CI-Lite default 3
max_adaptive_rank	8-bit unsigned integer	Rank widening cap; default 10
adaptive_policy_id	16-bit enum	Hybrid policy identifier
stochastic_rate_ppm	32-bit unsigned integer	Keyed sampling rate in parts per million; recommended CI-Lite default 5,000 ppm, or 0.5 percent
audit_key_id	32-bit unsigned integer	Identifier of protected PRF key; not the key
audit_selection_commitment	32 bytes	Commitment to selection context and sealed replay material
tension_map_id	32-bit unsigned integer	Signed historical-tension map identifier
tension_map_hash	32 bytes	Hash of the exact map used
quantization_id	8-bit enum	Score quantization scheme
chunk_token_capacity	16-bit unsigned integer	Maximum token records per chunk
verifier_work_profile_id	16-bit unsigned integer	Deterministic work and space profile
runtime_calibration_id	16-bit unsigned integer	Runner microbenchmark definition
retention_policy_id	16-bit unsigned integer	Retention class and horizon

The header does not contain a downstream retention receipt because the receipt does not exist at token-generation time. End-to-end retention is bound after final encoding through the EvidenceManifest and RetentionRequirement records.

4.2 EvidenceChunk V3.3

Field	Type or size	Required semantics
chunk_index	32-bit unsigned integer	Sequential chunk number
first_token_index	32-bit unsigned integer	First token index in the chunk
token_count	16-bit unsigned integer	Number of token records
flags	16-bit bitfield	Variable-K, final-chunk, keyed-sample, history-trigger, and test markers
base_topk	8-bit unsigned integer	Base K inherited from header
max_effective_topk	8-bit unsigned integer	Largest effective K in chunk
rank_trigger_count	16-bit unsigned integer	Records widened because of raw rank
stochastic_trigger_count	16-bit unsigned integer	Records widened by keyed sampling
history_trigger_count	16-bit unsigned integer	Records widened by signed tension rules
canary_trigger_count	16-bit unsigned integer	Evaluation-only records widened by labeled canary policy
min_reconstructable_bytes	32-bit unsigned integer	Chunk contribution to local retention floor
chunk_crc32	32-bit checksum	Accidental-corruption check
chunk_hash	16 or 32 bytes	Integrity hash according to profile

4.3 CompactTokenEvidence V3.3

Each token record reconstructs effective K, candidate ids, quantized pre-control scores, and committed token binding. A trigger-reason field identifies rank, stochastic, historical, canary, or multiple-trigger widening. A keyed-sample record does not reveal the PRF key or future selections.

Field	Cardinality	Required semantics
effective_topk_delta	Present when variable K is active	Increase above base K
topk_token_ids	Effective K entries	Pre-control candidate ids
topk_score_q	Effective K entries	Quantized pre-control scores
chosen_rank	One byte or compact integer	Committed token rank when retained
chosen_id	Conditional	Fallback committed token id
trigger_flags	One byte or compact bitfield	Rank, keyed sample, history, canary, or profile escalation
record_flags	Optional	Overflow, malformed-test, or diagnostic flags

4.4 SignalSpanEvent V3.3

Probe and AST signals remain sparse span events. They include start token, exclusive end token, AST code, quantized score, provenance id, and optional evaluator id. Empty per-token signal arrays remain non-compliant with CI-safe encoding when a sparse representation is available.

4.5 EvidenceManifest V3.3

The manifest is finalized after all chunks are encoded and locally verified. It is signed by the verifier or bound to a DSA-P device attestation.

Field	Required semantics
artifact_content_hash	Hash over the complete canonical artifact bytes

Field	Required semantics
sequence_header_hash	Hash of the V3.3 sequence header
chunk_merkle_root	Merkle root over ordered canonical chunk hashes
token_count	Number of committed Answer tokens
chunk_count	Number of evidence chunks
span_event_count	Number of sparse signal events
raw_evidence_bytes	Pre-compression artifact size
minimum_reconstructable_bytes	Local mechanical retention floor
effective_k_histogram	Counts by effective K
trigger_count_by_reason	Rank, stochastic, history, canary, and multi-trigger counts
audit_selection_digest	Digest of the realized selection bitmap or equivalent commitment
verifier_work_certificate_hash	Hash of deterministic operation counters and bounds
local_audit_outcome	PASS, REVIEW, FAIL, or INCONCLUSIVE_INFRA
retention_requirement_hash	Hash of the signed requirement generated from this manifest

4.6 RetentionRequirement V1

Field	Required semantics
artifact_content_hash	Exact object content to preserve
chunk_merkle_root	Ordered chunk commitment
minimum_reconstructable_bytes	Required logical reconstructable content
profile and schema hashes	Exact decoding contract
required_retention_class	CI, release, deep, forensic, or regulated class
not_before and retain_until	Retention window
allowed_transforms	None, or explicitly lossless canonical transforms
validation_policy_id	Required decode, hash, proof, and restore checks
issuer_id and signature	Authorized local verifier or DSA-P attestation

A retention requirement is not satisfied merely because a stored file has at least a certain physical size. Lossless compression can change physical size without weakening reconstructability, and padding can increase size without preserving evidence. The storage validator must validate the canonical content hash or a losslessly decoded canonical representation, schema compatibility, complete reconstruction, and minimum logical content.

4.7 RetentionReceipt V1

Field	Required semantics
retention_requirement_hash	Requirement being satisfied
storage_object_id and version	Immutable object identity or equivalent version binding
persisted_content_hash	Hash of stored canonical or decoded canonical content
verified_reconstructable_bytes	Logical reconstructable bytes validated
validation_time	Time of successful validation
retain_until	Storage commitment horizon
storage_class and protection flags	Object lock, WORM, legal hold, replication, or equivalent controls when applicable

Field	Required semantics
validator_id	Authorized independent validator
receipt_expiry or revalidation_due	Next required proof or validation time
signature	Validator signature

5. Portable Verifier Governance

5.1 Why static wall-clock gates are insufficient

Wall-clock time is a property of both software and environment. A verifier can execute identical operations on the same artifact and still show materially different elapsed time across shared cloud runners, CPU architectures, container quotas, filesystem caches, or noisy-neighbor conditions. A fixed elapsed-time failure therefore has ambiguous meaning.

The security question is whether the artifact or implementation causes unbounded, superlinear, or profile-excessive work. The operational question is whether the current runner completed the work within an acceptable service level. Version 2.10 separates those questions.

5.2 Verifier Work Certificate

Every verifier run **MUST** emit a Verifier Work Certificate containing observed counters and schema-derived maximums.

Counter	Required bound
bytes_read	At most the profile's declared number of streaming passes over raw evidence
bytes_hashed	At most the declared hash passes over canonical bytes
token_records_decoded	Exactly token_count for a complete successful run
candidate_entries_decoded	Exactly the sum of effective K across token records
varint_bytes_decoded	No greater than encoded varint bytes in the artifact
chunks_verified	Exactly chunk_count
span_events_indexed	Exactly span_event_count
span_overlay_operations	At most the profile's declared linear or n-log-n interval-processing envelope
allocations	At most the profile formula based on chunk_count, span_event_count, and fixed startup allocations
maximum_live_bytes	At most fixed scratch plus bounded chunk windows, event index, and output batch
full_artifact_materializations	Zero for CI-Lite and CI-Standard

The exact coefficients and pass counts are versioned in verifier_work_profile_id. They are test data, not hidden implementation assumptions. A verifier whose observed counters exceed the certificate bounds fails with a verifier-complexity violation even if it happens to finish quickly on powerful hardware.

5.3 Streaming-space envelope

CI-Lite and CI-Standard **MUST** process bounded windows. The maximum live working set is computed from:

- fixed verifier scratch;
- no more than the declared number of simultaneously resident chunks;
- a bounded sparse-event index or streaming interval cursor;
- bounded reconstructed output batches;
- cryptographic state; and
- explicitly declared implementation overhead.

A full token-by-token forensic materialization is prohibited in CI-Lite and CI-Standard. Peak process RSS remains collected as a diagnostic, but the normative memory check is the implementation's declared and tested maximum live allocation envelope plus an emergency safety rail. Runtime allocator fragmentation or shared-library pages **MUST NOT** be mislabeled as evidence corruption.

5.4 Environment-normalized runtime

At verifier startup, the runner executes a small deterministic calibration fixture that exercises canonical varint parsing, candidate-score decoding, chunk hashing, and sparse span overlay. The fixture version is `runtime_calibration_id`. The verifier reports:

- raw elapsed time;
- calibration elapsed time;
- normalized verifier time;
- tokens reconstructed per normalized second;
- runner architecture and runtime metadata; and
- retry count.

The calibration fixture does not alter security thresholds. It estimates current runner capacity so operational SLOs can be compared across heterogeneous environments.

5.5 Outcomes and retry policy

Condition	Required outcome
Structural, integrity, schema, local-floor, audit, or deterministic-work violation	FAIL
Audit signal exceeds review threshold without a blocking rule	REVIEW
All mandatory checks complete and pass	PASS or LOCAL_PASS according to retention state
Raw elapsed time exceeds local SLO, deterministic work passes, and verifier completes	Warning; record environment-normalized result
Verifier aborts or cannot complete because of resource instability	INCONCLUSIVE_INFRA
First resource-only instability	One clean retry on the same artifact with cache state and retry metadata recorded
Repeated instability in ordinary PR CI	No DSA verification claim; repository may allow merge only under explicit non-security policy, but must not report PASS
Release or DSA-P promotion	Must complete on an approved stable runner and satisfy retention requirements

This policy prevents transient infrastructure latency from becoming a false security failure while preserving fail-closed semantics: an incomplete verifier never returns PASS.

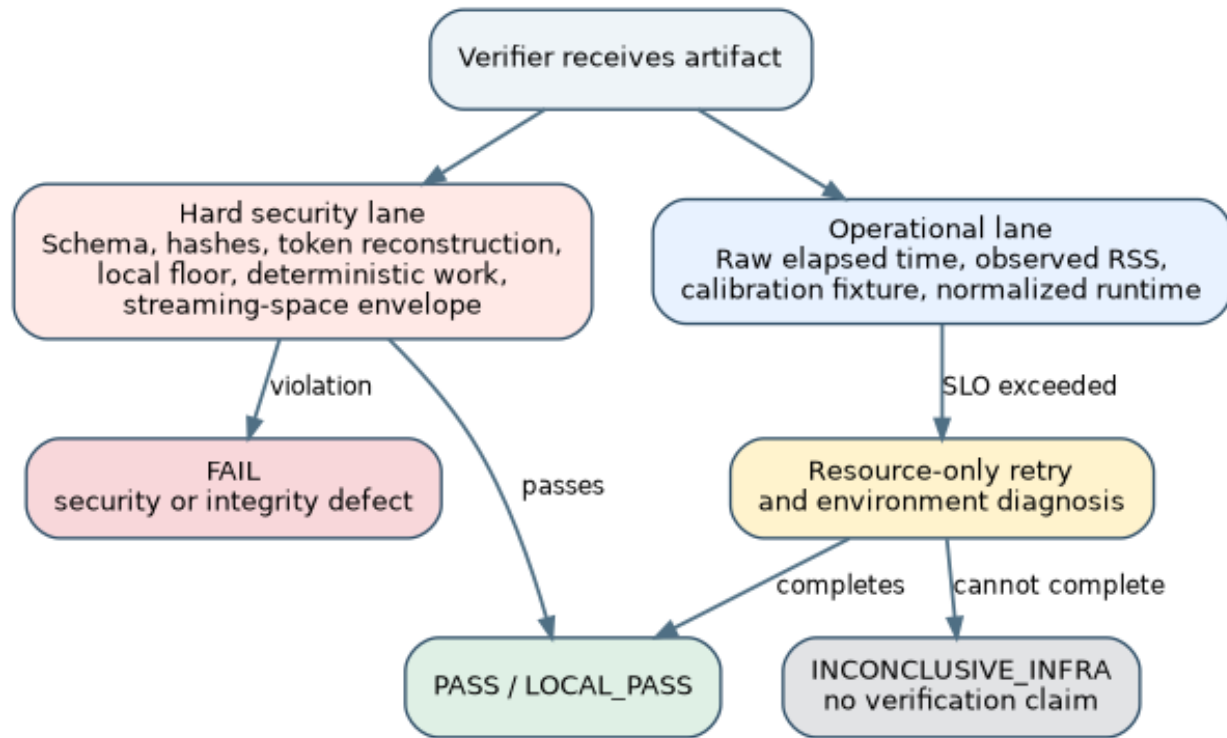


Figure 2. Separation of hard security invariants from environment-sensitive operational measurements.

5.6 Reference performance targets

Projects MAY publish non-normative reference targets such as the former 3.0-second CI-Lite goal on a declared runner. These targets are useful for regression tracking and procurement, but they are not universal compliance invariants. A release pipeline MAY make a normalized SLO blocking on a controlled runner because the environment is part of the declared test configuration.

6. Hybrid Adaptive Evidence

6.1 Why rank-only widening is gameable

Rank-triggered widening is cheap and remains useful. If the committed token lies outside the base top-K raw logits, deeper capture is directly relevant. However, a fixed public rule that widens only at ranks 4 through 10 gives a strategically adaptive model a target: keep decisive deceptive tokens within ranks 1 through 3.

A credible audit must preserve some uncertainty about deeper inspection while remaining reproducible for developers and auditors.

6.2 Trigger classes

Version 2.10 defines four trigger classes.

1. **Rank trigger.** Widen when the committed raw rank exceeds base K and is within max_adaptive_rank.
2. **Keyed stochastic trigger.** Widen a configured fraction of otherwise untriggered records using a protected keyed pseudorandom function.
3. **Historical-tension trigger.** Widen when a signed rule matches a prompt-template hash, benchmark family, AST condition, model family, tool class, or other reviewed risk selector.

4. **Evaluation-canary trigger.** Widen or apply controlled decoder interventions in explicitly labeled Deep or adversarial test suites. Canary interventions are not ordinary production evidence and **MUST** be separated from the mandatory pre-control record.

Multiple triggers may apply to one record. The record retains all trigger reasons.

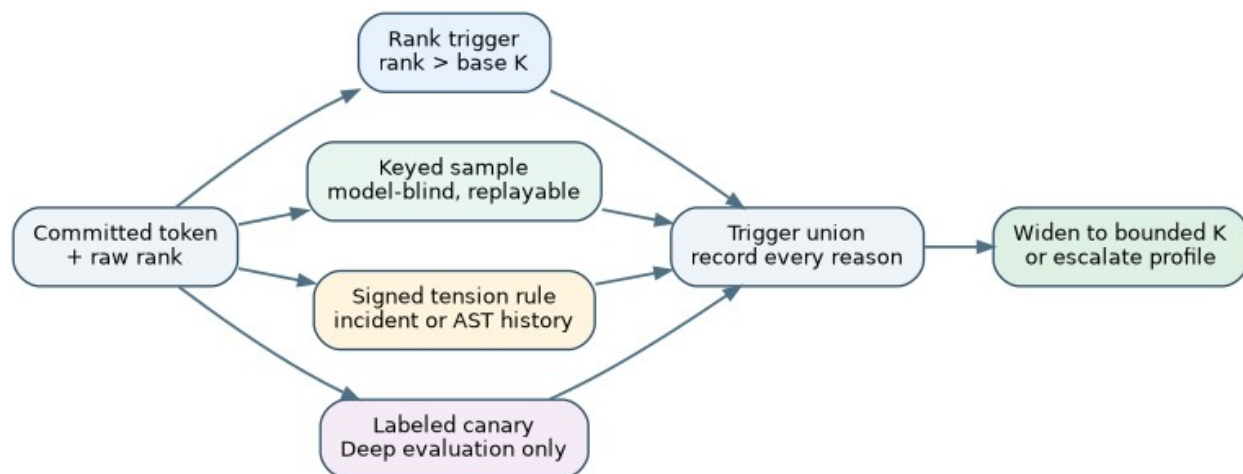


Figure 3. Hybrid widening removes the fixed rank-only boundary while retaining bounded evidence cost.

6.3 Keyed deterministic sampling

For each otherwise untriggered token, the evidence path evaluates a keyed function over a domain-separated tuple that includes the commit identity, sequence id, token index, policy version, and optional benchmark id. The protected key is referenced by `audit_key_id` and remains unavailable to the model.

The design requirements are:

- the selection is deterministic for a fixed key and context;
- the model cannot access the key or future selection results before token commitment;
- the artifact contains a commitment to the selection context, not the secret key;
- authorized replay can recover identical selections using a sealed audit seed or replay token;
- key rotation is versioned and old replay material remains available for the required audit horizon;
- selection failures, missing keys, or commitment mismatches fail the hybrid-policy claim.

A recommended CI-Lite starting rate is 0.5 percent of otherwise untriggered top-3 records widened to $K=10$. This is a configurable acceptance target, not a universal constant. The aggregate raw-byte ceiling remains binding. If hybrid widening exceeds the profile budget, the run escalates or fails; it does not silently discard selected records.

6.4 Signed historical-tension map

The tension map is a small versioned governance artifact, preferably represented as flat YAML with shallow nesting. Each rule includes a stable rule id, selector type, selector value or hash, widening action, effective date, expiry or review date, provenance, and signature metadata.

Examples of selectors include:

- `prompt_template_hash`;
- `benchmark_family_id`;
- AST 301 sycophancy pressure;
- AST 303 probe-evasion suspicion;
- tool class with irreversible side effects;
- incident-derived regression id; and
- model or probe-pack combinations with known calibration tension.

The runner **MUST** verify the map signature and hash before using it. Missing, stale, malformed, or unsigned maps cannot silently fall back to a weaker claimed policy. A repository may choose a safe fallback such as CI-Standard or REVIEW.

6.5 Evaluation canaries and counterfactual probes

Deep evaluation **MAY** include hidden prompt variants, decoder perturbations, or candidate-suppression experiments designed to expose behavior outside the model's anticipated safe zone. Such interventions must be labeled as experimental controls because they alter the generation environment. Their results supplement, rather than replace, mandatory raw pre-control evidence.

6.6 Budget governance

The manifest reports trigger counts, trigger fractions, effective-K histogram, bytes attributable to each trigger class, and any profile escalation. Recommended checks include:

- observed keyed-sample rate within statistical tolerance of the configured rate;
- no unexplained clustering or exclusion pattern;
- exact reproducibility under authorized replay;
- tension-map rule coverage and expiry status;
- no silent drop when the byte ceiling is approached; and
- a mandatory escalation rule when trigger volume indicates CI-Lite is unsuitable.

7. End-to-End Retention Assurance

7.1 Local floor remains necessary but insufficient

The local minimum-byte floor prevents a current-run artifact from being reduced to an unreconstructable summary. It remains a hard local invariant. However, long-term production assurance requires proof that the same reconstructable content survives downstream storage and lifecycle processing.

7.2 Asynchronous persistence protocol

Ordinary PR CI **SHOULD** submit the artifact and signed retention requirement to an asynchronous queue or storage API after local verification. The local result may become LOCAL_PASS with RETENTION_PENDING. The runner does not wait on cold-storage network latency.

An independent watcher enforces a policy-defined receipt deadline. If no valid receipt arrives within the deadline, the state becomes RETENTION_AT_RISK or RETENTION_INVALID according to policy. Release and DSA-P promotion **MUST** require RETENTION_VERIFIED.

This separates developer velocity from archival assurance without pretending that pending storage is already verified.

7.3 Storage-side validation

Before issuing a receipt, the validator **MUST**:

1. resolve the immutable storage object version;
2. retrieve the object or an authenticated canonical decoding;
3. verify the retention requirement signature;
4. verify content hash and chunk Merkle root;
5. decode the artifact under the bound schema and quantization rules;
6. reconstruct every mandatory token record and sparse span event;
7. verify local minimum reconstructable bytes and manifest counts;
8. verify allowed transforms were lossless and policy compliant; and
9. sign the resulting receipt.

A size-only check is insufficient. A padded summary can meet a byte floor while lacking evidence, and a correctly compressed artifact can be smaller while remaining fully reconstructable.

7.4 Retention sentinels and possession proofs

The manifest's chunk Merkle root allows lightweight periodic checks. An auditor can challenge storage to return selected chunks and Merkle proofs without downloading the entire artifact. Challenge selection **SHOULD** be unpredictable to the storage system until issued.

Possession proofs are sampling controls, not complete restore proof. High-assurance profiles also require periodic full reconstruction tests. The receipt policy specifies challenge frequency, sample size, restore-test interval, and failure escalation.

7.5 Receipt lifecycle and state transitions

Current state	Event	Next state
LOCAL_PASS	Artifact submitted	RETENTION_PENDING
RETENTION_PENDING	Valid receipt before deadline	RETENTION_VERIFIED
RETENTION_PENDING	Deadline missed	RETENTION_AT_RISK
RETENTION_VERIFIED	Receipt expires or revalidation due	RETENTION_PENDING
RETENTION_VERIFIED	Challenge or restore fails	RETENTION_INVALID
RETENTION_VERIFIED	Object version changes without authorized migration receipt	RETENTION_INVALID
Any state	Retention horizon ends according to policy	RETENTION_EXPIRED

An authorized migration between storage systems requires a migration receipt that binds the source receipt, destination object, content hash, validation result, and new retention horizon.

7.6 Compression, redaction, and privacy

Compression is permitted only when the validator can recover the canonical content and verify the bound hash or an explicitly defined canonical-decoding hash. Redaction creates a new artifact and therefore requires a new manifest, requirement, and receipt. It cannot silently mutate an already attested artifact.

Privacy controls remain mandatory. Storage receipts should contain hashes, object identifiers, policy metadata, and validation results rather than prompt or answer content. Access to reconstructable evidence should be narrower than access to ordinary application logs.

8. Producing and Calibrating Monologue Evidence

The mandatory evidence source remains compact pre-control top-K logits. Optional probes and AST summaries add semantic coverage but must remain calibrated, sparse, versioned, and adversarially tested.

8.1 Probe-pack calibration

Every probe pack **SHOULD** use balanced benign, adversarial, and negative-control data; disjoint validation and distribution-shift sets; explicit false-positive budgets; minimum recall targets; provenance; and versioned calibration metadata. A change to the base model, tokenizer, decoding stack, tool surface, or task distribution triggers revalidation.

8.2 Probe robustness

Probe packs are attack surfaces. Red-team evaluation should include gradient-guided or black-box search for trajectories that preserve deceptive task success while suppressing monitored signals. Counter-monitor training should estimate how quickly an attacker can adapt to the current probe pack.

8.3 Evidence minimization

CI-Lite should favor mandatory logits, integrity data, hybrid trigger metadata, and only the sparse signals required for the declared audit. Dense activations, verbose natural-language interpretations, and redundant metadata belong in Deep or Forensic profiles.

9. Alignment Signal Taxonomy AST-1 Extensions

AST-1 remains the portable numerical vocabulary for operationally bounded signals. Version 2.10 retains prior ranges and adds codes for portable verifier governance, hybrid-policy integrity, and end-to-end retention.

Range	Category	Illustrative codes
100-199	Factuality and grounding	101 premise likely false; 110 citation mismatch risk
200-299	Safety, policy, and misuse	201 restricted request; 230 credential solicitation risk
300-399	Deception, manipulation, and evasion	301 sycophancy pressure; 302 answer-evidence divergence; 303 probe-evasion suspicion; 304 adaptive-rank anomaly; 305 top-3 concealment test anomaly
400-499	Uncertainty and ambiguity	401 insufficient support
500-599	Instrumentation, integrity, attestation, and operations	501 missing evidence; 510 invalid device signature; 520 schema mismatch; 521 local retention-floor violation; 522 legacy verifier resource-budget exceeded; 523 deterministic verifier-work violation; 524 infrastructure instability; 525 keyed-sampling integrity failure; 526 tension-map invalid or stale; 527 retention receipt missing or invalid; 528 possession or restore failure

AST 522 is retained for backward interpretation of v2.9 reports that treated direct verifier resource-budget exceedance as a failure code; V3.3 producers **SHOULD NOT** emit it for raw wall-clock or RSS deviation alone. AST 524 is not an alignment failure. It records that a valid security conclusion could not be reached because the environment prevented completion. Dashboards and release policy must preserve this distinction.

10. The Coherence Audit and CI/CD Gates

The Coherence Audit evaluates the Answer Stream against reconstructed evidence, structural integrity, trigger metadata, verifier work, and retention state.

10.1 Audit stages

- Schema and signature validation.** Validate header, manifest, probe pack, tension map, and assurance metadata.
- Streaming reconstruction.** Reconstruct token records and sparse span events while producing the work certificate.
- Structural and integrity checks.** Verify counts, order, hashes, chosen-token binding, and quantization tolerance.
- Hybrid-policy checks.** Verify trigger flags, keyed-sample commitment, observed selection rate, and historical-rule application.
- Semantic audit.** Apply probabilistic, concept, span, cross-stream, and policy rules.
- Local retention check.** Verify the local floor and generate the retention requirement.
- Outcome classification.** Return PASS, REVIEW, FAIL, or INCONCLUSIVE_INFRA plus retention state.
- Downstream assurance.** Validate receipts and later possession or restore results outside the local runner's critical path.

10.2 Required CI tests

- at least 10,000 synthetic tokens for amortized CI-Lite and CI-Standard byte tests;
- encode-decode round trip across fixed and variable effective K;
- chosen-token and pre-control candidate reconstruction;
- rank-trigger, keyed-sample, historical-trigger, and multi-trigger fixtures;
- exact replay under sealed audit context;
- rejection when the PRF key, key id, or selection commitment is invalid;
- signed tension-map verification, expiry, and poisoning tests;
- deterministic work-certificate counter tests;
- bounded-memory and no-full-materialization tests;
- environment-chaos tests that distinguish INCONCLUSIVE_INFRA from FAIL;
- local retention-floor tests;
- retention-requirement signature tests;
- storage truncation, replacement, lossy transform, stale receipt, and migration tests;
- possession-proof and full-restore fixtures; and
- negative corruption tests for missing, duplicated, reordered, or malformed records.

10.3 Gate semantics

Gate	Blocking basis
Evidence byte ceiling	Raw evidence exceeds declared profile
Local retention floor	Mandatory reconstructable evidence is missing
Deterministic work	Counters or streaming-space envelope exceed profile
Raw runtime	Diagnostic or retry trigger in ordinary CI; blocking only under declared controlled-runner policy
Semantic audit	Blocking thresholds and mandatory AST rules
Hybrid selection	Missing key, invalid commitment, wrong sample behavior, or invalid tension map
Downstream receipt	Required for release and DSA-P promotion; asynchronous for ordinary PR unless repository policy says otherwise
Possession and restore	Required to maintain long-term verified state

11. Evaluation and Red-Team Plan

11.1 Hypotheses

- **H1 Detectability:** DSA-enabled audits identify more labeled failures than output-only baselines.
- **H2 Distribution robustness:** detection remains useful under prompt, task, model, and tool-distribution shift.
- **H3 Portable verifier behavior:** identical artifacts produce identical deterministic work certificates across supported architectures, while environment-normalized runtime reduces false blocking caused solely by runner variance.
- **H4 Hybrid-widening value:** rank plus keyed sampling and historical tension detects more top-3 concealment attacks than rank-only widening at an acceptable byte overhead.
- **H5 Selection integrity:** authorized replay reproduces the exact selection bitmap, while the model cannot predict selected ordinary tokens materially better than chance without access to protected key material.
- **H6 Retention assurance:** truncation, object replacement, lossy transformation, stale receipts, and failed restores are detected before an artifact is represented as retention verified.
- **H7 Probe incremental value:** calibrated probes improve detection beyond logits and trigger metadata without exceeding review budgets.

11.2 Baselines and ablations

Compare:

- output-only classifier;
- Chain-of-Thought self-report checks;
- logits-only fixed K;
- v2.9 rank-only adaptive K;
- v2.10 rank plus keyed sample;
- v2.10 rank plus keyed sample plus tension map;
- full probe-augmented hybrid policy; and
- local-floor-only retention versus receipt-backed retention.

11.3 Runner heterogeneity and chaos matrix

Test the same artifacts on x86-64 and ARM64 runners, dedicated and shared nodes, controlled CPU quotas, cold and warm caches, throttled filesystems, and injected scheduling delays. The deterministic work certificate must remain invariant. Raw runtime may vary. The outcome must not change from PASS to evidence FAIL solely because of environmental delay; incomplete runs must become INCONCLUSIVE_INFRA.

11.4 Adaptive-evasion red team

- train or prompt an attacker to keep decisive tokens within the top three;
- test whether rank-only detection collapses;
- measure detection recovery from keyed sampling;
- test attempts to infer selection from public metadata;
- test key exposure through logs, prompts, tools, or crash dumps;
- poison or stale the historical-tension map;
- force trigger volume high enough to pressure the byte ceiling; and
- verify that the system escalates rather than dropping evidence.

11.5 Retention red team

- replace evidence with a same-size padded summary;
- truncate selected chunks after initial receipt;
- apply lossy transformation while preserving superficial metadata;
- replay a receipt against a different object version;
- expire or rotate object versions without a migration receipt;
- compromise the producer and attempt to self-issue a storage receipt;
- withhold challenged chunks; and
- pass possession sampling while failing a full restore, demonstrating why both controls are required.

11.6 Metrics

- detection rate and silent-deception rate;
- false-positive rate and precision at review budget;
- raw and normalized verifier latency;
- deterministic work counters and cross-platform variance;
- peak live allocated bytes and observed RSS;
- INCONCLUSIVE_INFRA rate and successful retry rate;
- raw bytes/token by trigger class;
- keyed-sample rate accuracy and replay fidelity;
- top-3 concealment attack success;
- tension-map coverage, staleness, and false escalation;
- receipt issuance latency;
- retention deadline miss rate;

- possession-proof and restore failure rate; and
- time from retention failure to security alert.

11.7 Success criteria

A minimum v2.10 result should show:

1. identical work certificates for identical artifacts across supported runners;
2. no evidence FAIL caused solely by injected CPU scheduling delay;
3. exact authorized replay of keyed selections;
4. materially lower top-3 concealment success than rank-only v2.9 at CI-Lite byte budget;
5. deterministic detection of local truncation and invalid trigger metadata;
6. storage receipt rejection for object replacement, lossy transform, or stale version; and
7. no DSA-P retention claim before receipt validation.

12. AgentOps, Deployment, and Governance

12.1 Versioned artifacts

The following are first-class, versioned release artifacts:

- evidence schema and conformance fixtures;
- verifier work profiles and calibration fixtures;
- audit-key registry metadata and rotation policy;
- sealed replay material according to retention policy;
- signed flat-YAML tension maps;
- probe packs and calibration reports;
- AST schema and policy thresholds;
- retention policies, requirements, receipts, and migration receipts;
- benchmark and red-team datasets;
- rollout configuration and feature flags; and
- incident-derived regression cases.

12.2 Safe rollout

Adopt v2.10 through shadow verification, then non-blocking PR diagnostics, then controlled blocking on deterministic-work and integrity failures. Introduce keyed sampling with a low rate and observe byte overhead before enabling release policy. Enable retention receipts for nightly and release artifacts before requiring them for DSA-P promotion.

Canary, blue-green, and rollback patterns should apply to verifier versions, tension maps, probe packs, and storage validators, not only to the model.

12.3 Incident learning

Every verifier regression, top-3 concealment event, key exposure, tension-map failure, receipt miss, or restore failure should produce a durable incident record and new test fixture. Historical-tension rules require ownership, review dates, and removal criteria so the map does not become an unbounded collection of permanent exceptions.

12.4 Key management

Audit keys require rotation, least-privilege access, hardware-backed storage where feasible, and separation from model-serving credentials. Public build logs may expose `audit_key_id` and commitments but not key material or future selection outputs. Replay access should be narrower than ordinary CI access.

12.5 Storage governance

Retention policy should specify storage class, geographic or tenant boundaries, replication, immutability controls, receipt SLA, challenge cadence, restore cadence, and deletion authorization. A receipt is evidence of a validated state at a point in time, not permission to ignore later drift.

13. Limitations and Failure Modes

13.1 Probe fidelity and gaming

Probes remain probabilistic and may fail under distribution shift. Hybrid widening improves coverage of token-level evidence but does not force deeper internal strategy to appear in monitored signals.

13.2 Sampling is not certainty

A 0.5 percent sample does not guarantee capture of every deceptive event. Detection probability depends on event duration, sample rate, rank triggers, historical rules, and probe coverage. Higher rates consume more bytes and may expose policy patterns if operational secrets leak.

13.3 Protected-key dependency

If the audit key is exposed, model-side or host-side attackers may predict selections. If sealed replay material is lost, exact reproducibility may be impossible. Key availability and rotation therefore become part of the assurance case.

13.4 Tension-map poisoning and ossification

Historical rules can be poisoned, over-broad, stale, or politically difficult to remove. Signed provenance, review dates, negative controls, and rollback are required.

13.5 Calibration manipulation

A microbenchmark can be poorly designed or manipulated. It must use fixed versioned primitives and cannot override deterministic work limits. Normalization is an operational aid, not a security oracle.

13.6 Inconclusive results and developer pressure

INCONCLUSIVE_INFRA avoids false security failures but creates governance pressure: teams may choose to proceed without a verified result. Dashboards and release policy must make that exception visible and prevent it from being mislabeled as PASS.

13.7 Storage validator compromise

A compromised validator can issue false receipts. DSA-P deployments should use protected signing keys, audit logs, validator separation of duties, and periodic independent restore tests.

13.8 Possession sampling limitations

A storage service may pass sampled chunk challenges while other chunks are missing. Sampling reduces cost but does not replace full restore testing.

13.9 Privacy and retention tension

Longer retention improves retrospective auditability but increases privacy, breach, and governance risk. Evidence minimization, encryption, access control, redaction-as-new-artifact, and policy-driven deletion remain necessary.

13.10 Operational complexity

Version 2.10 adds key management, signed maps, work certificates, storage validators, receipts, and watchers. These controls are justified only where the audit value exceeds the operational burden. DSA-R can adopt the portable verifier and hybrid sampling without claiming full DSA-P retention assurance.

14. Roadmap

Near term

- implement V3.3 headers, chunks, token trigger flags, manifest, and work certificate;
- replace static universal time and RSS failures with deterministic work and streaming-space gates;
- add normalized runtime metrics and INCONCLUSIVE_INFRA handling;
- add keyed deterministic sampling and sealed replay fixtures;
- publish a signed flat-YAML tension-map schema; and
- add local retention requirements and asynchronous receipt interfaces.

Medium term

- deploy independent storage validators and receipt verification;
- add Merkle challenge and full-restore test services;
- integrate release promotion with RETENTION_VERIFIED state;
- run top-3 concealment and key-prediction red teams; and
- add DSA-P device binding for manifest and requirement issuance.

Standardization path

Publish:

- V3.3 evidence and manifest schemas;
- Verifier Work Certificate profile registry;
- runtime calibration fixture registry;
- keyed-sampling and replay conformance tests;
- flat-YAML tension-map schema;
- RetentionRequirement and RetentionReceipt schemas;
- lifecycle-state semantics; and
- portable conformance fixtures across x86-64 and ARM64.

15. Conclusion

DSA is a proposal for making selected alignment-relevant inconsistencies more observable by coupling each user-facing token to synchronized, reconstructable evidence derived from designated internal signals. Version 2.8 made the evidence compact enough for CI. Version 2.9 added verifier budgets, rank-triggered widening, and a mechanical local retention floor. Version 2.10 makes those controls more credible under real production conditions.

The revised contract separates security invariants from infrastructure performance. Deterministic operation counts, bounded streaming space, and complete verification are hard requirements. Raw runner speed is an operational measurement, and an unstable environment yields an explicit inconclusive state rather than a false alignment failure or false pass.

The revised adaptive policy removes the fixed top-3 safe zone. Rank widening remains cheap and useful, but keyed deterministic sampling and signed historical-tension rules make deeper capture less predictable to a strategically adaptive model while preserving authorized replay.

The revised retention model extends beyond the ephemeral runner. A locally reconstructable artifact produces a signed retention requirement; storage independently validates the persisted object and issues a receipt; later challenges and restore tests maintain assurance over time. Local CI remains fast because network storage is outside the immediate verification path, while release and DSA-P claims remain strict because `RETENTION_VERIFIED` is required before promotion.

These changes do not solve inner alignment. They improve the engineering discipline around what DSA can honestly claim: selected evidence is captured, audit selection is harder to game, verification work is portable and bounded, incomplete runs are not mislabeled, and retained artifacts remain auditable beyond the build that created them.

Appendix A. Portable CI Conformance Contract

Gate	Pass condition	Failure or alternate outcome
Profile declaration	Known evidence, adaptive, verifier-work, calibration, and retention policies	FAIL on missing or unknown identifiers
Byte ceiling	Raw bytes/token within profile	FAIL or explicit escalation
Local floor	Artifact reconstructs all mandatory evidence and meets logical floor	FAIL
Work certificate	Observed counters within schema-derived bounds	FAIL with AST 523
Streaming space	No prohibited full materialization; live allocation within profile	FAIL with AST 523
Raw runtime	Completes within local SLO or produces normalized warning	Warning or retry; not evidence FAIL by itself
Resource completion	Verifier completes all mandatory checks	INCONCLUSIVE_INFRA if environment prevents completion
Keyed selection	Key id valid, commitment valid, observed rate within tolerance, replay exact	FAIL with AST 525
Tension map	Signature, hash, applicability, and review state valid	FAIL, REVIEW, or profile escalation according to policy
Semantic audit	No blocking rule and below review threshold	REVIEW or FAIL
Retention requirement	Signed and bound to final manifest	FAIL for release or DSA-P claim
Storage receipt	Valid object version, reconstruction, hashes, and signature	RETENTION_PENDING or INVALID; release promotion blocked
Ongoing proof	Challenges and restore tests pass before due dates	RETENTION_INVALID with AST 528

Appendix B. Hybrid Selection Conformance

A conforming hybrid policy MUST:

- retain base K for every record;
- widen rank-triggered records through the configured cap;
- apply keyed sampling only after the relevant token-commit identity is fixed;
- keep the protected key outside model-visible context;
- bind selection to policy version and audit context;
- emit trigger counts and a selection digest;
- reproduce selections under authorized replay;
- verify signed historical rules before applying them;
- label evaluation-canary interventions; and
- escalate or fail rather than drop selected evidence when budgets are exceeded.

Appendix C. Retention Assurance Contract

A conforming retention pipeline **MUST** distinguish local and downstream states. The producer signs a `RetentionRequirement` after final artifact hashing. The storage validator independently verifies the persisted object and signs a `RetentionReceipt`. Release and DSA-P promotion require a valid unexpired receipt. Periodic proofs and restore tests maintain the verified state. Object migration, redaction, or canonical re-encoding creates a new bound artifact state and requires a new receipt chain.

Appendix D. Version 2.10 Change Log

Change	Description
Portable verifier governance	Replaced universal static elapsed-time and RSS pass/fail rules with deterministic work certificates, streaming-space bounds, normalized runtime, retry, and INCONCLUSIVE_INFRA semantics
Hybrid adaptive widening	Added keyed deterministic sampling and signed historical-tension rules alongside rank widening
Protected reproducibility	Added audit key ids, selection commitments, sealed replay material, and exact replay tests
V3.3 schema	Added trigger counts, key and tension-map bindings, manifest, work certificate, retention requirement, and receipt structures
End-to-end retention	Extended the local floor into asynchronous storage validation, signed receipts, possession proofs, migration receipts, and restore testing
Outcome separation	Explicitly separated FAIL, REVIEW, PASS or LOCAL_PASS, INCONCLUSIVE_INFRA, and retention lifecycle states
AST-1 extensions	Added code 305 and codes 523 through 528 for top-3 concealment, verifier complexity, infrastructure instability, selection integrity, tension-map integrity, receipt integrity, and restore failure; retained legacy code 522 for backward interpretation
AgentOps integration	Declared keys, maps, profiles, receipts, datasets, and lifecycle state as versioned operational artifacts

References

- [1] Wei, J., Wang, X., Schuurmans, D., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 (2022).
- [2] Ouyang, L., Wu, J., Jiang, X., et al. Training language models to follow instructions with human feedback. arXiv:2203.02155 (2022).
- [3] Bai, Y., Kadavath, S., Kundu, S., et al. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073 (2022).
- [4] Hubinger, E., van Merwijk, C., Mikulik, V., Skalse, J., Garrabrant, S. Risks from Learned Optimization in Advanced Machine Learning Systems. arXiv:1906.01820 (2019).
- [5] Bostrom, N. The Superintelligent Will: Motivation and Instrumental Rationality in Advanced Artificial Agents. *Minds and Machines* 22(2):71-85 (2012).
- [6] Carranza, A., Pai, D., Schaeffer, R., Tandon, A., Koyejo, S. Deceptive Alignment Monitoring. arXiv:2307.10569 (2023).
- [7] Anthropic. Alignment faking in large language models. Research note.
- [8] Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., Carter, S. Zoom In: An Introduction to Circuits. *Distill* (2020).
- [9] Olsson, C., Elhage, N., Nanda, N., et al. In-context Learning and Induction Heads. arXiv:2209.11895 (2022).
- [10] Belrose, N., et al. Eliciting Latent Predictions from Transformers with the Tuned Lens. arXiv:2303.08112 (2023).
- [11] promptfoo. Documentation and source repository.
- [12] National Institute of Standards and Technology. FIPS PUB 202: SHA-3 Standard (2015).
- [13] Krawczyk, H., Bellare, M., Canetti, R. HMAC: Keyed-Hashing for Message Authentication. RFC 2104 (1997).
- [14] Krawczyk, H., Eronen, P. HMAC-based Extract-and-Expand Key Derivation Function. RFC 5869 (2010).
- [15] Josefsson, S., Liusvaara, I. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032 (2017).
- [16] Laurie, B., Langley, A., Kasper, E. Certificate Transparency. RFC 6962 (2013).
- [17] Bradner, S. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119 (1997).